

Ex.No:3 Wireshark

Date:

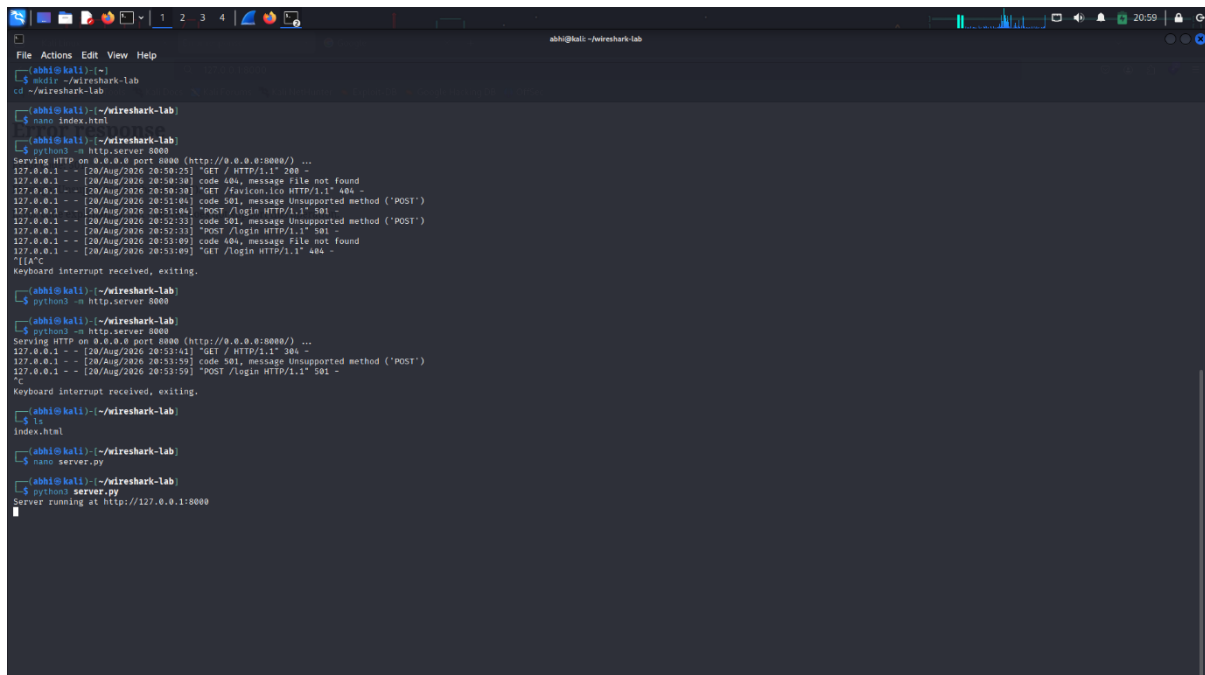
Password Capturing

Wireshark can capture not only passwords but any type of information transmitted over the network: usernames, email addresses, personal information, etc. As long as we can capture network traffic, Wireshark can sniff passing passwords.

In sniffing can include passwords for various protocols such as HTTP, FTP, Telnet, etc. the captured data can be used to troubleshoot network problems, but can also be used maliciously to gain unauthorized access to sensitive information.

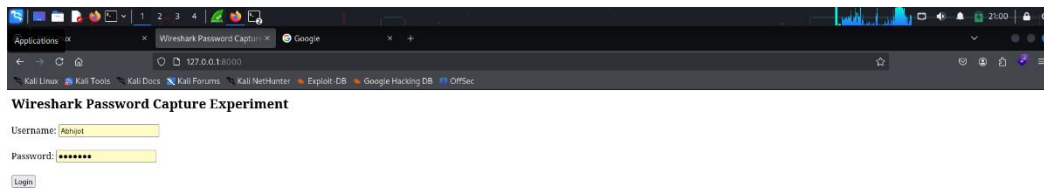
So, here we will see how we can capture the password using the Wireshark network capture analyzer. and see the outputs of the following steps.

Step 1: First of all, open your Wireshark tool in your window or in Linux virtual machine. and start capturing the network. suppose I am capturing my wireless fidelity.



```
abhi@kali: ~$ sudo wireshark-lab
abhi@kali: ~$ nano index.html
abhi@kali: ~$ python3 http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
127.0.0.1 - - [20/Aug/2026 20:50:25] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [20/Aug/2026 20:50:30] code 404, message File not found
127.0.0.1 - - [20/Aug/2026 20:50:30] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [20/Aug/2026 20:51:04] code 501, message Unsupported method ('POST')
127.0.0.1 - - [20/Aug/2026 20:51:04] "POST /login HTTP/1.1" 501 -
127.0.0.1 - - [20/Aug/2026 20:52:33] code 501, message Unsupported method ('POST')
127.0.0.1 - - [20/Aug/2026 20:52:33] "POST /login HTTP/1.1" 501 -
127.0.0.1 - - [20/Aug/2026 20:53:09] code 404, message File not found
127.0.0.1 - - [20/Aug/2026 20:53:09] "GET /login HTTP/1.1" 404 -
Keyboard interrupt received, exiting.
abhi@kali: ~$ python3 http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
127.0.0.1 - - [20/Aug/2026 20:53:41] "GET / HTTP/1.1" 304 -
127.0.0.1 - - [20/Aug/2026 20:53:59] code 501, message Unsupported method ('POST')
127.0.0.1 - - [20/Aug/2026 20:53:59] "POST /login HTTP/1.1" 501 -
Keyboard interrupt received, exiting.
abhi@kali: ~$ ls
index.html
abhi@kali: ~$ nano server.py
abhi@kali: ~$ python3 server.py
Server running at http://127.0.0.1:8000
```

Step 2: After starting the packet capturing we will go to the website and login the credential on that website as you can see in the image.



Step 3: Now after completing the login credential we will go and capture the password in Wireshark. for that we have to use some filter that helps to find the login credential through the packet capturing.

Step 4: Wireshark has captured some packets but we specifically looking for HTTP packets. so in the display filter bar we use some command to find all the captured HTTP packets. as you can see in the below image the green bar where we apply the filter.

http



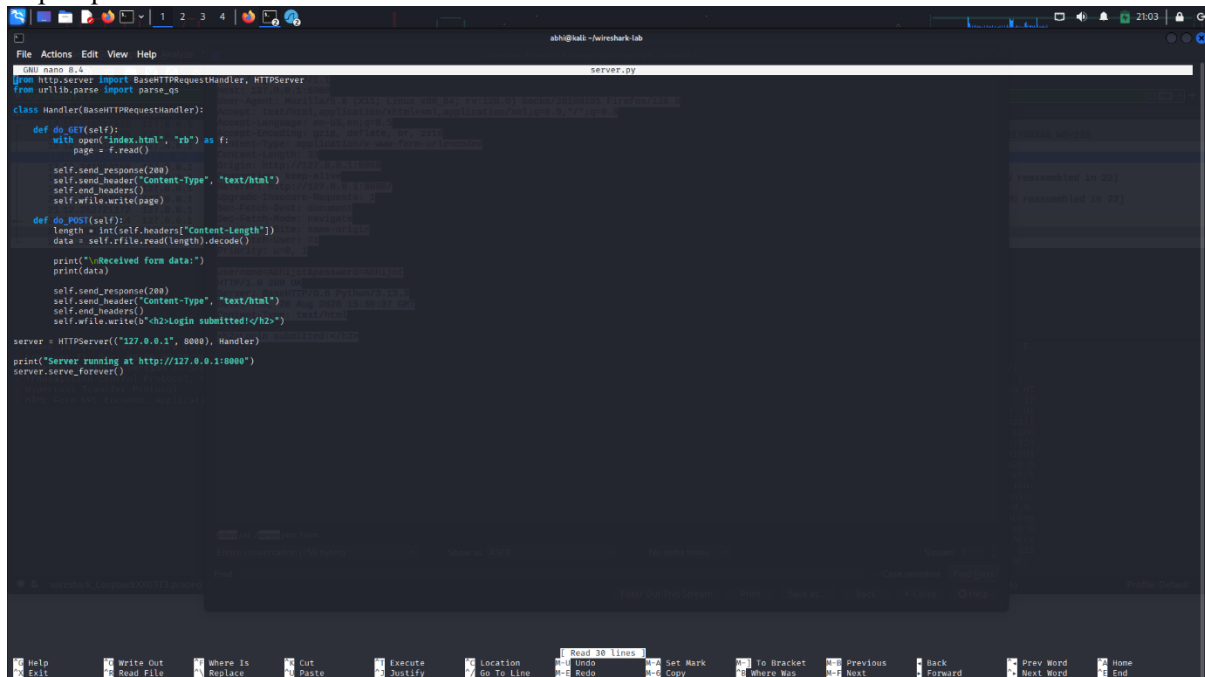
Step 5: So there are some HTTP packets are captured but we specifically looking for form data that the user submitted to the website. for that, we have a separate filter

As we know that there are main two methods used for submitting form data from web pages like login forms to the server. the methods are-

- GET
- POST

Step 6: So firstly for knowing the credential we use the first method and apply the filter for the GET methods as you can see below.

`http.request.method == "GET"`



```
server.py
from http.server import BaseHTTPRequestHandler, HTTPServer
from urllib.parse import parse_qs

class Handler(BaseHTTPRequestHandler):
    def do_GET(self):
        with open("index.html", "rb") as f:
            page = f.read()

        self.send_response(200)
        self.send_header("Content-Type", "text/html")
        self.end_headers()
        self.wfile.write(page)

    def do_POST(self):
        length = int(self.headers["Content-Length"])
        data = self.rfile.read(length).decode()

        print("\nReceived form data:")
        print(data)

        self.send_response(200)
        self.send_header("Content-Type", "text/html")
        self.end_headers()
        self.wfile.write(b"<h2>Login submitted!</h2>")

server = HTTPServer(("127.0.0.1", 8080), Handler)

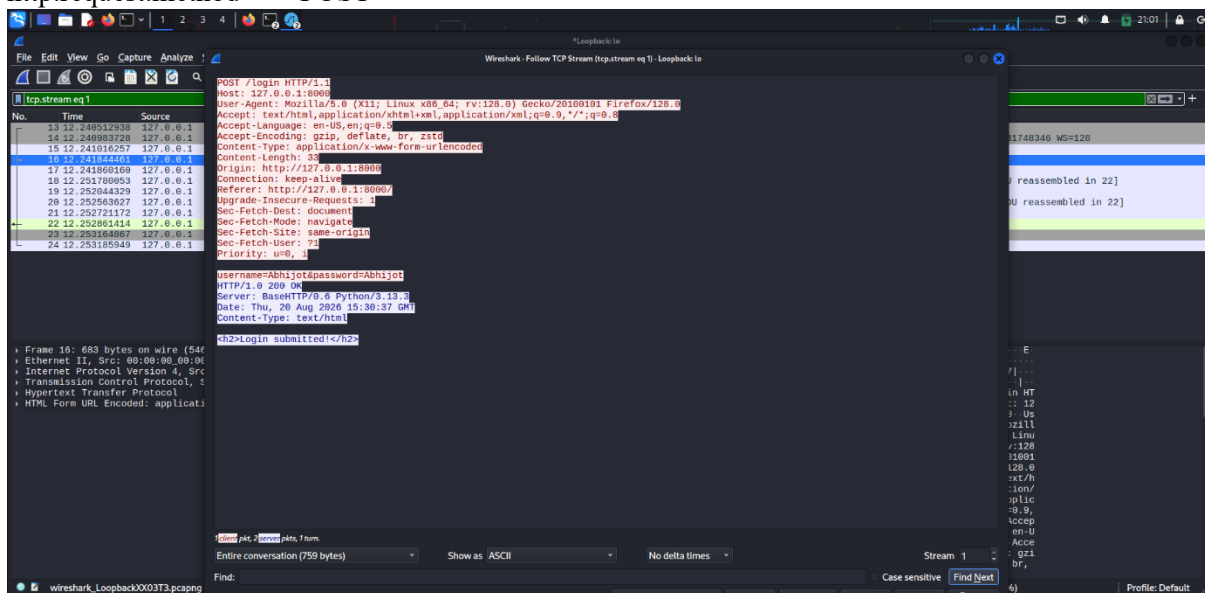
print("Server running at http://127.0.0.1:8080")
server.serve_forever()
```

GET method

As you can see in the image there are two packets where the login page was requested with a GET request as well, but there is no form data submitted with a GET request.

Step 7: Now after checking the GET method if we didn't find the form data, then we will try the POST method for that we will apply the filter on Wireshark as you can see.

`http.request.method == "POST"`



As you can see we have a packet with form data click on the packet with user info and the application URL encoded. and click on the down-

HTML form URL Encoded where the login credential is found. login credential as it is the same that we filed on the website in step 2.

Form item: "uname" = "Tonystark_44"

Form item: "pass" = "tony@1234"